

Documentação Técnica Completa

Projeto: Análise Sociodemográfica e Econômica Nacional — IBGE 2022/2023

Sumário

1. [Etapa 1 — Coleta, Limpeza e Preparação dos Dados](#)
 - 1.1 Ambiente e justificativa das ferramentas
 - 1.2 Diagnóstico do arquivo bruto
 - 1.3 Os cinco problemas encontrados e como foram resolvidos
 - 1.4 Pipeline de limpeza linha a linha
 - 1.5 Resultado e validação final
 2. [Etapa 2 — Análise e Automação via SQL](#)
 - 2.1 Ingestão no banco de dados
 - 2.2 Desafio 1: Densidade Demográfica
 - 2.3 Desafio 2: Agregação Macrorregional
 - 2.4 Desafio 3: Subquery Dinâmica de Frota
 - 2.5 Desafio 4: Vulnerabilidade Social com AND
 - 2.6 O requisito de classificação automática (CASE WHEN)
 3. [Etapa 3 — Dashboard Interativo HTML](#)
 - 3.1 Por que HTML e não Power BI ou Tableau
 - 3.2 Arquitetura técnica do dashboard
 - 3.3 Como cada gráfico responde ao que o projeto pediu
 - 3.4 Sistema de navegação e interatividade
-

Etapa 1 — Coleta, Limpeza e Preparação dos Dados {#etapa-1}

1.1 Ambiente e justificativa das ferramentas

O trabalho de limpeza foi inteiramente conduzido em **Python 3**, utilizando exclusivamente as bibliotecas **pandas** e **re** (módulo de expressões regulares da biblioteca padrão). A escolha não foi arbitrária — ela reflete as necessidades reais do arquivo fornecido.

Por que Python e não Excel?

O Excel seria a ferramenta óbvia para quem visualiza isso como "limpeza de planilha", mas ele traz três limitações críticas para este projeto específico:

Primeiro, **o Excel não gerencia encoding de forma controlada**. Ao abrir um arquivo `latin-1` no Excel, ele tenta autodetectar o charset e frequentemente o faz incorretamente, corrompendo silenciosamente os acentos sem emitir nenhum alerta. O Python, ao contrário, força o programador a declarar o encoding explicitamente no parâmetro `encoding='latin1'` do `pd.read_csv()`, tornando a decisão auditável e documentada.

Segundo, **o Excel não diferencia tipos de dados com precisão suficiente**. Uma coluna como `area_km2` que contém `"164173,429"` (string com vírgula decimal) seria importada pelo Excel como texto ou como número com arredondamento imprevisível. No pandas, a transformação é explícita: `str.replace(',', '.').astype(float)` — o desenvolvedor sabe exatamente o que está acontecendo em cada linha.

Terceiro, **o Excel não é reproduzível**. Qualquer transformação feita manualmente numa planilha é invisível para quem receberá o arquivo depois. O script Python é, em si, a documentação do que foi feito — cada decisão de limpeza vira código que pode ser revisado, auditado e re-executado.

Por que pandas especificamente?

O pandas é a biblioteca de referência absoluta para ETL em dados tabulares dentro do ecossistema Python. Suas vantagens concretas neste projeto foram:

- `pd.read_csv()` aceita parâmetros de encoding e delimitador diretamente, sem pré-processamento do arquivo
- `DataFrame.rename()` com dicionário permite mapear nomes antigos para novos em uma única operação declarativa
- `Series.str.replace()` aplica substituições vetorizadas em toda a coluna sem loops manuais
- `Series.astype()` converte tipos de dados com validação automática — se qualquer valor não puder ser convertido, o Python lança um erro imediatamente, forçando o desenvolvedor a tratar o problema

1.2 Diagnóstico do arquivo bruto

Antes de qualquer transformação, o arquivo foi inspecionado em três níveis:

Nível 1 — Bytes brutos

```
with open('ibge_raw.csv', 'rb') as f:
    raw_bytes = f.read(200)
print(raw_bytes[:100].hex())
```

A leitura dos bytes brutos (`'rb'` — read binary) sem nenhuma decodificação aplicada permite identificar o encoding real do arquivo antes de qualquer tentativa de interpretação. Um arquivo UTF-8 puro tem bytes na faixa `0x00–0x7F` para caracteres ASCII e sequências específicas de dois ou três bytes para caracteres fora dessa faixa. Um arquivo `latin-1` usa bytes únicos na faixa `0x80`

–0xFF para os caracteres acentuados — o byte 0xE3, por exemplo, representa o ã nesse charset. Identificar isso antes de abrir o arquivo evita o erro silencioso de abrir tudo com UTF-8 e substituir caracteres inválidos por ? ou \ufffd.

Nível 2 — Tentativa de leitura com parâmetros errados (diagnóstico intencional)

```
# Tentativa 1: encoding errado
df_wrong = pd.read_csv('ibge_raw.csv', encoding='utf-8')
# Resultado esperado: UnicodeDecodeError no byte 0xE3 (ã de "Área")

# Tentativa 2: encoding correto, delimitador errado
df_comma = pd.read_csv('ibge_raw.csv', encoding='latin1')
print(df_comma.shape)
# Resultado: (27, 1) – 27 linhas mas APENAS 1 COLUNA
# Todas as colunas colapsadas porque o delimitador padrão (vírgula) não existe no arquivo
```

Testar deliberadamente com parâmetros incorretos é uma técnica de diagnóstico que confirma as hipóteses antes de agir. O resultado (27, 1) é a prova concreta de que o delimitador não é vírgula.

Nível 3 — Leitura correta e inventário de problemas

```
df = pd.read_csv('ibge_raw.csv', encoding='latin1', delimiter=';')
print(df.dtypes)
print(df.isnull().sum())
```

Com os parâmetros corretos, o DataFrame foi carregado com shape (27, 14) — 27 estados, 14 colunas. A inspeção dos tipos (dtypes) revelou o problema central: cinco colunas numéricas estavam tipadas como object (string) em vez de float64, porque o pandas não consegue interpretar vírgula como separador decimal automaticamente.

1.3 Os cinco problemas encontrados e como foram resolvidos

Problema 1 — Encoding latin-1 (ISO-8859-1)

Causa raiz: O IBGE historicamente gera arquivos com o charset latin-1, padrão da Microsoft para Windows em português brasileiro antes da adoção generalizada do UTF-8. O byte 0xE3 representa ã, 0xE7 representa ç, 0xED representa í — todos mapeados como bytes únicos nesse charset, enquanto UTF-8 os codificaria como sequências de dois bytes.

Como foi resolvido:

```
df = pd.read_csv('ibge_raw.csv', encoding='latin1', delimiter=';')
```

O parâmetro encoding='latin1' instrui o pandas a usar o codec ISO-8859-1 ao decodificar cada byte do arquivo. Após a limpeza, o arquivo de saída foi exportado em UTF-8 — o padrão moderno — para garantir compatibilidade com qualquer banco de dados ou ferramenta subsequente:

```
df.to_csv('ibge_censo_limpo.csv', index=False, encoding='utf-8', sep=';')
```

Problema 2 — Delimitador ponto-e-vírgula

Causa raiz: O ponto-e-vírgula como delimitador é padrão em arquivos CSV gerados por aplicações em países que usam vírgula como separador decimal. Como o Brasil usa 1.234,56 (ponto como separador de milhar, vírgula como decimal), o Excel e outros softwares brasileiros adotam ; como delimitador para não confundir com os decimais.

Como foi resolvido:

```
df = pd.read_csv('ibge_raw.csv', encoding='latin1', delimiter=';')
```

O parâmetro `delimiter=';'` (equivalente a `sep=';'`) define explicitamente o caractere separador de colunas. Sem ele, o pandas assume , por padrão e trata toda a linha como uma única string.

Problema 3 — Decimais no padrão brasileiro (vírgula)

Causa raiz: Cinco colunas contêm valores com vírgula decimal no formato brasileiro:

- `area_km2`: "164173,429"
- `densidade_demografica`: "5,06"
- `idh`: "0,71"
- `receitas_realizadas_mil`: "6632883,10836"
- `despesas_empenhadas_mil`: "6084416,8063"

O pandas lê esses valores como strings (object) porque "164173,429" não é um número válido em nenhuma localização padrão do Python. Tentar converter diretamente com `astype(float)` lançaria `ValueError`.

Como foi resolvido:

```
float_cols = ['area_km2', 'densidade_demografica', 'idh',  
             'receitas_realizadas_mil', 'despesas_empenhadas_mil']  
  
for col in float_cols:  
    df[col] = df[col].str.replace(',', '.', regex=False).astype(float)
```

A operação é encadeada em dois passos:

1. `str.replace(',', '.', regex=False)` — substitui a vírgula decimal por ponto em toda a coluna de forma vetorizada. O parâmetro `regex=False` é importante aqui: sem ele, a vírgula seria interpretada como um padrão de expressão regular, o que seria desnecessariamente custoso e potencialmente perigoso se houvesse caracteres especiais nos dados.
2. `.astype(float)` — converte a Series inteira de object para float64. Se qualquer valor ainda não puder ser convertido (por exemplo, um campo vazio ou texto), o pandas lança `ValueError` imediatamente, forçando o tratamento explícito.

Problema 4 — Nomes de colunas incompatíveis com SQL

Causa raiz: Os nomes originais das colunas vinham do notebook do Kaggle com três categorias de

problemas distintas:

```
# Categoria A: Tags HTML embutidas
'IDH <span>Índice de desenvolvimento humano</span> [2021]'

# Categoria B: Encoding corrompido (? no lugar de í)
'Matri?culas no ensino fundamental - matrículas [2021]'

# Categoria C: Símbolos especiais incompatíveis com SQL
'Área Territorial - km² [2022]' # km² com caractere especial
'Receitas realizadas - R$ (×1000) [2017]' # R$, ×, parênteses
'UF [-]' # colchetes e hífen
```

Nenhum desses nomes pode ser usado diretamente em SQL sem aspas delimitadoras, o que torna as queries verbosas e propensas a erros. O padrão SQL exige identificadores sem espaços, sem acentos e sem caracteres especiais para uso sem escape.

Como foi resolvido:

```
rename_map = {
    'UF [-]': 'uf',
    'Código [-]': 'codigo',
    'codigo_uf',
    'Gentílico [-]': 'gentilico',
    'Governador [2023]': 'governador',
    'Capital [2010]': 'capital',
    'Área Territorial - km² [2022]': 'area_km2',
    'População residente - pessoas [2022]': 'populacao',
    'Densidade demográfica - hab/km² [2022]': 'densidade_demografica',
    'Matri?culas no ensino fundamental - matrículas [2021]': 'matriculas_ensino_fundamental',
    'IDH <span>Índice de desenvolvimento humano</span> [2021]': 'idh',
    'Receitas realizadas - R$ (×1000) [2017]': 'receitas_realizadas_mil',
    'Despesas empenhadas - R$ (×1000) [2017]': 'despesas_empenhadas_mil',
    'Rendimento mensal domiciliar per capita - R$ [2023]': 'rendimento_per_capita',
    'Total de veículos - veículos [2022]': 'total_veiculos'
}

df = df.rename(columns=rename_map)
```

O uso de um dicionário de mapeamento manual é intencional. Uma abordagem automática (como `str.lower().str.replace(' ', '_')`) não removeria as tags HTML nem os símbolos especiais de forma confiável. O mapeamento explícito garante que cada nome novo seja deliberado e semanticamente correto.

A convenção adotada foi **snake_case** sem acentos — `rendimento_per_capita` em vez de `rendimento_mensal_domiciliar_per_capita_r$_[2023]`. Essa convenção é o padrão de mercado para nomes de colunas em bancos de dados relacionais.

Problema 5 — Coluna *regiao* ausente

Causa raiz: O dataset original do IBGE organiza os dados por UF mas não inclui a dimensão de macrorregião. Para o Desafio 2 do projeto, a query precisa executar `GROUP BY regiao` — se a coluna não existir na tabela, a query falha com erro de coluna inexistente.

Como foi resolvido:

```
regioes_map = {
    'Norte': ['Acre', 'Amapá', 'Amazonas', 'Pará',
              'Rondônia', 'Roraima', 'Tocantins'],
    'Nordeste': ['Alagoas', 'Bahia', 'Ceará', 'Maranhão', 'Paraíba',
                 'Pernambuco', 'Piauí', 'Rio Grande do Norte', 'Sergipe'],
    'Centro-Oeste': ['Distrito Federal', 'Goiás',
                     'Mato Grosso', 'Mato Grosso do Sul'],
    'Sudeste': ['Espírito Santo', 'Minas Gerais',
                'Rio de Janeiro', 'São Paulo'],
    'Sul': ['Paraná', 'Rio Grande do Sul', 'Santa Catarina']
}

# Inverte o dicionário: {uf: regiao} para cada UF em cada lista
uf_to_regiao = {uf: reg for reg, ufs in regioes_map.items() for uf in ufs}

df['regiao'] = df['uf'].map(uf_to_regiao)
```

A técnica usada aqui é uma **dict comprehension de inversão** seguida de `.map()`. O dicionário original está organizado como `{regiao: [lista_de_ufs]}` — útil para leitura humana. Para aplicar ao DataFrame, precisamos da forma invertida: `{uf: regiao}`. A compreensão `{uf: reg for reg, ufs in regioes_map.items() for uf in ufs}` percorre cada região, depois cada UF dentro dela, e cria o mapeamento inverso em uma única expressão.

O método `.map()` do pandas aplica esse dicionário como função de lookup vetorizada — para cada valor na coluna `uf`, ele busca o valor correspondente no dicionário e preenche a nova coluna `regiao`. Se alguma UF não estiver no dicionário (erro de digitação, por exemplo), o resultado seria `NaN`, que seria capturado pela verificação de nulos no passo seguinte.

1.4 Pipeline de limpeza linha a linha

O pipeline completo, na ordem de execução:

```
import pandas as pd

# PASSO 1: Carregamento com parâmetros corretos
df = pd.read_csv(
    'ibge_raw.csv',
    encoding='latin1',    # charset real do arquivo IBGE
    delimiter=';'         # delimitador padrão IBGE/Brasil
)
# Estado após este passo: shape (27, 14), tipos mistos

# PASSO 2: Renomeação de colunas → snake_case SQL-friendly
df = df.rename(columns=rename_map)
# Estado após este passo: 14 colunas com nomes limpos

# PASSO 3: Criação da coluna 'regiao'
df['regiao'] = df['uf'].map(uf_to_regiao)
# Estado após este passo: shape (27, 15)
```

```
# PASSO 4: Conversão decimal BR → EN e cast para float
for col in ['area_km2', 'densidade_demografica', 'idh',
            'receitas_realizadas_mil', 'despesas_empenhadas_mil']:
    df[col] = df[col].str.replace(',', '.', regex=False).astype(float)
# Estado após este passo: 5 colunas convertidas de object para float64

# PASSO 5: Padronização de strings
df['uf'] = df['uf'].str.strip()
df['governador'] = df['governador'].str.strip().str.title()
df['capital'] = df['capital'].str.strip()
df['gentilico'] = df['gentilico'].str.strip().str.lower()
# Estado após este passo: texto padronizado

# PASSO 6: Reordenação das colunas
col_order = ['uf', 'regiao', 'codigo_uf', 'gentilico', 'governador',
              'capital', 'area_km2', 'populacao', 'densidade_demografica',
              'matriculas_ensino_fundamental', 'idh', 'receitas_realizadas_mil',
              'despesas_empenhadas_mil', 'rendimento_per_capita',
              'total_veiculos']
df = df[col_order]

# PASSO 7: Exportação
df.to_csv('ibge_censo_limpo.csv', index=False, encoding='utf-8', sep=';')
```

1.5 Resultado e validação final

O arquivo limpo `ibge_censo_limpo.csv` tem as seguintes características verificadas:

Atributo	Antes	Depois
Shape	(27, 14)	(27, 15)
Encoding	latin-1	UTF-8
Delimitador saída	;	;
Colunas object indevidas	5 (área, IDH, etc.)	0
Valores nulos	0	0
Coluna <code>regiao</code> presente	Não	Sim
Nomes SQL-safe	0 de 14	15 de 15

Tipos de dados finais confirmados:

```
uf                str (object)
regiao            str (object)
codigo_uf         int64
gentilico          str (object)
governador        str (object)
capital           str (object)
area_km2          float64 ← convertido de string
populacao         int64
densidade_demografica float64 ← convertido de string
matriculas_ensino_fundamental int64
idh               float64 ← convertido de string
receitas_realizadas_mil float64 ← convertido de string
despesas_empenhadas_mil float64 ← convertido de string
rendimento_per_capita int64
```

total_veiculos	int64
----------------	-------

Etapa 2 — Análise e Automação via SQL {#etapa-2}

2.1 Ingestão no banco de dados

O CSV limpo foi ingerido em um banco **SQLite** via Python:

```
import sqlite3, pandas as pd

df = pd.read_csv('ibge_censo_limpo.csv', sep=';', encoding='utf-8')
conn = sqlite3.connect('ibge_censo.db')
df.to_sql('censo', conn, if_exists='replace', index=False)
```

Por que SQLite?

SQLite armazena o banco inteiro em um único arquivo `.db`, sem necessidade de servidor, daemon, usuário ou senha. Ele é compatível com DB Browser for SQLite (ferramenta visual gratuita), com DBeaver e com Python nativamente. Para um projeto acadêmico onde o objetivo é demonstrar SQL puro — e não administração de infraestrutura — é a escolha mais pragmática. As queries escritas são 100% compatíveis com PostgreSQL e MySQL sem modificação.

O método `df.to_sql()` inferiu automaticamente os tipos SQL a partir dos tipos pandas:

- `int64` → `INTEGER`
- `float64` → `REAL`
- `object (str)` → `TEXT`

2.2 Desafio 1: Densidade Demográfica

O que o projeto pediu: operação aritmética entre população e área, alias inteligível, ordenação decrescente.

O que foi implementado:

```
SELECT
    uf,
    regiao,
    populacao,
    ROUND(area_km2, 2) AS area_km2,
    ROUND(populacao / area_km2, 2) AS densidade_calculada,
    CASE
        WHEN (populacao / area_km2) >= 100 THEN 'Alta Densidade'
        WHEN (populacao / area_km2) >= 20  THEN 'Média Densidade'
        WHEN (populacao / area_km2) >= 5   THEN 'Baixa Densidade'
        ELSE                                'Muito Baixa Densidade'
    END AS classificacao_densidade
FROM censo
ORDER BY densidade_calculada DESC;
```

Análise cláusula por cláusula:

`ROUND(populacao / area_km2, 2) AS densidade_calculada` — a divisão `populacao / area_km2` é uma operação aritmética pura dentro do SQL. Como `populacao` é `INTEGER` e `area_km2` é `REAL`, o SQLite promove automaticamente o resultado para `REAL` (ponto flutuante). O `ROUND( . . . , 2)` arredonda para duas casas decimais — suficiente para a precisão necessária em hab/km². O alias `densidade_calculada` é o "pseudônimo inteligível" que o projeto exigiu.

`ORDER BY densidade_calculada DESC` — o `ORDER BY` referencia o alias definido no `SELECT`. Isso é possível porque o SQLite (e a maioria dos SGBDs modernos) resolve aliases do `SELECT` antes de aplicar o `ORDER BY`. O `DESC` garante que o Distrito Federal (489,06 hab/km²) apareça na primeira linha.

Resultado: 27 linhas ordenadas. Distrito Federal (489,06), Rio de Janeiro (402,49) e São Paulo (178,92) lideram. Amazonas (2,53), Roraima (2,84) e Mato Grosso (4,19) fecham a lista.

2.3 Desafio 2: Agregação Macrorregional

O que o projeto pediu: agrupamento por região geográfica, média aritmética simultânea de IDH e renda.

O que foi implementado:

```
SELECT
    regioao,
    COUNT(uf)                                AS total_estados,
    ROUND(AVG(idh), 3)                       AS media_idh,
    ROUND(AVG(rendimento_per_capita), 2)     AS media_rendimento_per_capita,
    CASE
        WHEN AVG(idh) >= 0.750 THEN 'IDH Alto'
        WHEN AVG(idh) >= 0.700 THEN 'IDH Médio-Alto'
        WHEN AVG(idh) >= 0.650 THEN 'IDH Médio'
        ELSE                          'IDH Baixo'
    END AS status_idh,
    CASE
        WHEN AVG(rendimento_per_capita) >= 2000 THEN 'Renda Alta'
        WHEN AVG(rendimento_per_capita) >= 1500 THEN 'Renda Média'
        ELSE                          'Renda Baixa'
    END AS status_renda
FROM censo
GROUP BY regioao
ORDER BY media_idh DESC;
```

Análise cláusula por cláusula:

`GROUP BY regioao` — essa cláusula é o mecanismo central da query. Ela instrui o banco a particionar as 27 linhas em grupos, onde cada grupo contém todos os estados de uma mesma região. O resultado é 5 grupos: Norte (7 estados), Nordeste (9), Centro-Oeste (4), Sudeste (4), Sul (3).

`AVG(idh)` e `AVG(rendimento_per_capita)` — as funções de agregação operam sobre cada grupo individualmente. `AVG(idh)` para o grupo "Norte", por exemplo, soma os IDHs de Acre, Amapá, Amazonas, Pará, Rondônia, Roraima e Tocantins e divide por 7. O mesmo acontece independentemente para cada região.

`COUNT(uf)` — conta quantas linhas (estados) existem em cada grupo. Serve para contextualizar as

médias: uma média de 7 estados tem peso diferente de uma média de 3.

`ROUND(AVG(idh), 3)` — o IDH é normalmente expresso com 3 casas decimais (0,756), então `ROUND(..., 3)` preserva a precisão convencional do índice PNUD.

O ponto crítico aqui é que o `CASE WHEN` dentro do `SELECT` pode referenciar `AVG(idh)` diretamente, sem necessidade de subquery — o SGBD calcula a agregação e a usa tanto no valor exibido quanto na lógica de classificação.

Resultado: 5 linhas. Sul (0,756 / R\$2.624), Sudeste (0,754 / R\$2.259), Centro-Oeste (0,751 / R\$2.439) com IDH Alto e Renda Alta. Norte (0,699 / R\$1.363) e Nordeste (0,686 / R\$1.169) com IDH Médio e Renda Baixa.

2.4 Desafio 3: Subquery Dinâmica de Frota

O que o projeto pediu: estados com frota acima da média nacional; proibido usar valores hardcoded; média calculada dinamicamente via subquery aninhada.

O que foi implementado:

```
SELECT
    uf,
    regioao,
    total_veiculos,
    ROUND((SELECT AVG(total_veiculos) FROM censo), 0) AS
media_nacional,
    ROUND(total_veiculos - (SELECT AVG(total_veiculos) FROM censo), 0) AS
excedente_da_media,
    CASE
        WHEN total_veiculos > (SELECT AVG(total_veiculos) FROM censo) * 3
        THEN 'Muito Acima da Média'
        WHEN total_veiculos > (SELECT AVG(total_veiculos) FROM censo)
        THEN 'Acima da Média'
    END AS status_frota
FROM censo
WHERE total_veiculos > (SELECT AVG(total_veiculos) FROM censo)
ORDER BY total_veiculos DESC;
```

Análise do mecanismo de subquery:

A subquery (`SELECT AVG(total_veiculos) FROM censo`) aparece quatro vezes na query. Cada ocorrência é tecnicamente uma query completa e independente que o banco executa em tempo de execução. O resultado — neste caso, aproximadamente 4.022.899 — é calculado dinamicamente a partir dos dados reais da tabela, não de um valor fixo no código.

Por que isso importa? Se os dados fossem atualizados (um novo Censo, por exemplo), a mesma query produziria automaticamente resultados corretos com a nova média, sem nenhuma modificação no código. Um valor hardcoded como `WHERE total_veiculos > 4022899` quebraria silenciosamente com dados novos.

Subquery no WHERE vs. no SELECT:

No `WHERE total_veiculos > (SELECT AVG(total_veiculos) FROM censo)` — a subquery age como valor de comparação no filtro. O banco primeiro resolve a subquery (obtém 4.022.899), depois filtra as linhas da query externa.

No `SELECT ROUND((SELECT AVG(total_veiculos) FROM censo), 0) AS media_nacional` — a mesma subquery aparece como expressão de coluna. Isso faz com que o valor da média nacional seja exibido em cada linha do resultado, facilitando a comparação visual no relatório.

Resultado: 8 estados. São Paulo (35.091.000) classificado como "Muito Acima da Média" (mais de 3× a média). Os outros 7 — Minas Gerais, Rio de Janeiro, Paraná, Rio Grande do Sul, Santa Catarina, Bahia e Goiás — como "Acima da Média".

2.5 Desafio 4: Vulnerabilidade Social com AND

O que o projeto pediu: múltiplos operadores lógicos na cláusula de restrição, concomitância de duas condições.

O que foi implementado:

```
SELECT
  uf,
  regioao,
  rendimento_per_capita,
  matriculas_ensino_fundamental,
  CASE
    WHEN rendimento_per_capita < 1000 THEN 'Vulnerabilidade Crítica'
    WHEN rendimento_per_capita < 1200 THEN 'Alta Vulnerabilidade'
    ELSE 'Vulnerabilidade Moderada'
  END AS nivel_vulnerabilidade,
  CASE
    WHEN matriculas_ensino_fundamental > 1000000 THEN 'Alta Demanda Escolar'
    WHEN matriculas_ensino_fundamental > 500000 THEN 'Média Demanda
Escolar'
    ELSE 'Baixa Demanda
Escolar'
  END AS demanda_escolar
FROM censo
WHERE rendimento_per_capita < 1500
  AND matriculas_ensino_fundamental > 200000
ORDER BY rendimento_per_capita ASC;
```

Análise do operador AND:

O AND na cláusula WHERE implementa a interseção lógica entre dois conjuntos:

- Conjunto A: estados com `rendimento_per_capita < 1500` → 14 estados
- Conjunto B: estados com `matriculas_ensino_fundamental > 200000` → ~20 estados
- Interseção $A \cap B$: estados que satisfazem **ambas** simultaneamente → 12 estados

Estados como Amapá e Roraima ficam fora porque, apesar da baixa renda, têm menos de 200.000 matrículas (populações pequenas). Estados do Sul e Sudeste ficam fora porque, apesar de terem muitas matrículas, têm renda acima de R\$1.500.

Os dois CASE WHEN do SELECT aplicam subcategorização adicional **dentro** dos resultados filtrados, classificando cada estado por grau de criticidade da renda e por volume de demanda escolar independentemente.

Resultado: 12 estados, todos do Nordeste (10) e Norte (2). Maranhão com R\$945 e 1.064.699 matrículas é o caso mais crítico. Nenhum estado do Sul, Sudeste ou Centro-Oeste atendeu às duas condições simultaneamente.

2.6 O requisito de classificação automática (CASE WHEN)

Este é o ponto que diferencia o projeto de um exercício básico de SQL. A docente pediu explicitamente que **"o próprio sistema gere a interpretação de forma automática, baseada em parâmetros"** e que **"o banco de dados mastigue essa classificação"**.

O mecanismo técnico para isso é o `CASE WHEN . . . THEN . . . END` — uma estrutura de controle de fluxo dentro do SQL que avalia uma expressão para cada linha e retorna um valor com base em qual condição for verdadeira primeiro.

Em vez de retornar apenas números brutos e deixar o analista classificar manualmente em Excel, as queries foram construídas para que cada linha já venha com uma coluna de status calculada pelo SGBD:

- `classificacao_densidade`: 'Alta Densidade', 'Média Densidade', 'Baixa Densidade', 'Muito Baixa Densidade'
- `status_idh`: 'IDH Alto', 'IDH Médio-Alto', 'IDH Médio', 'IDH Baixo'
- `status_renda`: 'Renda Alta', 'Renda Média', 'Renda Baixa'
- `status_frota`: 'Muito Acima da Média', 'Acima da Média'
- `nivel_vulnerabilidade`: 'Vulnerabilidade Crítica', 'Alta Vulnerabilidade', 'Vulnerabilidade Moderada'
- `demanda_escolar`: 'Alta Demanda Escolar', 'Média Demanda Escolar', 'Baixa Demanda Escolar'

Isso transforma o output SQL de um conjunto de números em um relatório classificado — exatamente o que a docente descreveu como "relatório automatizado".

Etapa 3 — Dashboard Interativo HTML {#etapa-3}

3.1 Por que HTML e não Power BI ou Tableau

Essa decisão merece uma explicação técnica detalhada, porque vai contra o planejamento inicial do trabalho.

O problema com Power BI neste contexto:

Power BI é uma ferramenta de BI corporativa que exige instalação de desktop (Windows only nativamente) ou licença Pro para publicação online. O arquivo gerado (`.pbix`) é binário e proprietário — não pode ser aberto sem o Power BI Desktop instalado. Para um trabalho acadêmico que precisa ser entregue e avaliado, isso significa que a docente precisaria ter o Power BI instalado para ver o resultado. Além disso, o Power BI conecta bem a bancos PostgreSQL e MySQL, mas a

conexão com SQLite exige um driver ODBC adicional e configuração manual.

O problema com Tableau:

Tableau Public é gratuito mas publica os dashboards na nuvem do Tableau — o arquivo local (.twbx) também é proprietário. A curva de aprendizado para criar visualizações customizadas com classificações como as que o projeto pede (colorir barras pelo `status_idh`, por exemplo) é significativa. E, assim como o Power BI, requer que quem recebe o arquivo tenha a ferramenta instalada.

Por que HTML foi a escolha correta:

Um arquivo HTML é universalmente acessível — qualquer navegador moderno o abre sem instalação de nada. O arquivo pode ser entregue por email, ZIP, Google Drive, ou publicado em qualquer servidor. Não tem dependências além de uma conexão internet para carregar a biblioteca Chart.js, que pesa menos de 200KB.

Do ponto de vista técnico, HTML + JavaScript permite controle total sobre cada pixel do layout, o que é necessário para reproduzir fielmente as classificações automáticas geradas pelo SQL como elementos visuais (cores, categorias, tooltips). Em Power BI, criar uma lógica de coloração condicional baseada nos campos de status exigiria o uso de DAX (linguagem proprietária). No HTML, é JavaScript puro.

A biblioteca escolhida: Chart.js

Chart.js é uma biblioteca open-source de visualização de dados com CDN público. A decisão de usá-la em vez de D3.js foi deliberada: o D3 oferece mais controle mas exige construção manual de cada elemento SVG; o Chart.js oferece tipos de gráficos prontos (bar, scatter, line) que são exatamente o que os quatro desafios precisam, com opções de tooltip, eixos duplos e coloração condicional por array.

3.2 Arquitetura técnica do dashboard

O dashboard é um único arquivo HTML autossuficiente estruturado em três camadas:

Camada 1 — Estrutura (HTML semântico)

```
<!-- KPIs globais no topo -->
<div class="kpi-g">
  <div class="kpi-c"> <!-- IDH médio nacional: 0,717 --> </div>
  <div class="kpi-c"> <!-- UFs em vulnerabilidade: 12 --> </div>
</div>

<!-- Sistema de abas para os 4 desafios -->
<nav class="nav">
  <a class="tab" href="#s1">D1 — Densidade</a>
  <a class="tab" href="#s2">D2 — Regional</a>
  <a class="tab" href="#s3">D3 — Frota</a>
  <a class="tab" href="#s4">D4 — Vulnerabilidade</a>
</nav>

<!-- Seções de cada desafio -->
<section id="s1"> <!-- SQL syntax-highlighted + gráfico + insight --> </section>
```

Camada 2 — Estilo (CSS com variáveis do sistema)

Em vez de cores hardcoded, o CSS usa variáveis como `var(--color-text-primary)`, `var(--color-background-secondary)` e `var(--color-border-info)`. Isso garante que o dashboard respeita automaticamente o tema claro ou escuro do ambiente onde é exibido, sem nenhuma lógica adicional de detecção de tema.

Camada 3 — Dados e interatividade (JavaScript)

Os dados dos 27 estados foram embutidos diretamente no JavaScript como arrays de objetos — replicando o resultado de cada query SQL:

```
// Resultado do Desafio 1 embutido como array JS
const diall = [
  {uf: 'Distrito Federal', r: 'Centro-Oeste', d: 489.06, c: 'A'},
  {uf: 'Rio de Janeiro', r: 'Sudeste', d: 402.49, c: 'A'},
  // ... 25 estados restantes
];
// c: 'A'=Alta, 'M'=Média, 'B'=Baixa, 'X'=Muito Baixa
```

A propriedade `c` (categoria) é o equivalente JavaScript da coluna `classificacao_densidade` gerada pelo `CASE WHEN` no SQL. Ela é usada para definir a cor de cada barra:

```
backgroundColor: data.map(d => colorMap[d.c] + 'CC')
// 'A' → '#E24B4A' (vermelho), 'M' → '#EF9F27' (laranja)
// 'B' → '#1D9E75' (verde), 'X' → '#888780' (cinza)
```

3.3 Como cada gráfico responde ao que o projeto pediu

Desafio 1 → Gráfico de barras horizontal

Um ranking de 27 itens é naturalmente melhor visualizado em barras horizontais — os rótulos das UFs cabem sem rotação no eixo Y. A ordenação decrescente (topo = maior densidade) reflete exatamente o `ORDER BY densidade_calculada DESC` da query SQL. As quatro cores representam as quatro categorias do `CASE WHEN`. O filtro por região (botões `Norte` / `Nordeste` / `etc.`) foi adicionado como recurso interativo extra — ao clicar, o gráfico reconstrói com apenas os estados da região selecionada, permitindo comparação dentro de cada macrorregião.

Desafio 2 → Gráfico de barras agrupadas com eixo duplo

IDH e Renda são grandezas em escalas completamente diferentes (0,686–0,756 vs. R\$1.169–R\$2.624). Plotá-las no mesmo eixo Y tornaria uma das séries ilegível. A solução é o eixo Y duplo: IDH no eixo esquerdo (verde, escala 0,65–0,82) e Renda no eixo direito (laranja, escala R\$0–R\$3.100). O tooltip ao passar o mouse exibe tanto o valor numérico quanto o status classificado pelo `CASE WHEN` ('IDH Alto', 'Renda Baixa', etc.).

Desafio 3 → Gráfico de barras verticais com linha de média

O uso de dois tipos de gráfico sobrepostos (barras + linha) é a visualização mais clara para mostrar "quais estados estão acima de um limiar". A linha tracejada vermelha na altura de 4.022.899 é o equivalente visual da subquery (`SELECT AVG(total_veiculos) FROM censo`) — ela é dinâmica no SQL e é um valor fixo derivado dos dados no gráfico. São Paulo recebe cor vermelha (#E24B4A) por ser classificado como "Muito Acima da Média", enquanto os outros 7 recebem azul

(#378ADD).

Desafio 4 → Scatter plot (dispersão)

O scatter plot é o tipo de gráfico mais adequado para mostrar a relação entre duas variáveis numéricas contínuas — renda (eixo X) e matrículas (eixo Y) — onde cada ponto representa um estado. A posição espacial de cada ponto codifica simultaneamente os dois critérios do AND da query. A cor do ponto codifica o nível de vulnerabilidade do CASE WHEN. O tooltip ao passar o mouse sobre cada ponto exibe o nome do estado, a renda exata, o número de matrículas e o nível classificado.

3.4 Sistema de navegação e interatividade

KPIs animados no topo:

Os quatro cards numéricos (27 UFs, IDH 0,717, Renda R\$1.731, 12 vulneráveis) são preenchidos com animação `easeOut` via `requestAnimationFrame`:

```
function animC(id, target, fmt, dur) {
  const t0 = performance.now();
  (function step(now) {
    const p = Math.min(1, (now - t0) / dur);
    const e = 1 - Math.pow(1 - p, 3); // curva easeOut cúbica
    el.textContent = fmt(e * target);
    if (p < 1) requestAnimationFrame(step);
  })(t0);
}
```

A curva $1 - (1-p)^3$ acelera no início e desacelera no final, criando a animação de contagem que "freia" ao chegar no valor final — padrão visual de dashboards profissionais.

Sistema de abas:

A navegação por abas usa `scrollIntoView({behavior: 'smooth'})` para deslizar até cada seção ao clicar, mantendo o contexto do dashboard visível. A aba ativa recebe a classe `on` que aplica a borda inferior e muda a cor do texto.

Filtro por região no Desafio 1:

```
window.filterD1 = function(btn) {
  const r = btn.dataset.r;
  buildD1(r === 'Todos' ? d1all : d1all.filter(d => d.r === r));
};
```

A função `buildD1()` destrói o `Chart.js` existente (`C1.destroy()`) e reconstrói um novo com os dados filtrados. A altura do canvas é recalculada dinamicamente — `Math.max(320, data.length * 40 + 80)px` — para que o gráfico nunca fique comprimido independentemente de quantas barras estão visíveis.

Conclusão técnica

O projeto percorreu um pipeline completo de engenharia de dados:

```
Arquivo bruto (latin-1, ;, vírgula decimal, HTML nos headers)
  ↓ Python/pandas: 7 passos de limpeza
CSV limpo (UTF-8, ;, float64, snake_case, +coluna regioao)
  ↓ sqlite3: to_sql()
Banco SQLite (tabela censo, 27 linhas × 15 colunas)
  ↓ SQL puro: 4 queries com CASE WHEN + subquery
Resultados classificados automaticamente
  ↓ Chart.js/HTML: 4 tipos de gráfico + KPIs + filtros
Dashboard interativo entregável
```

Cada decisão técnica — de `encoding='latin1'` no `read_csv` ao eixo Y duplo no Chart.js — foi determinada pelos dados reais e pelos requisitos específicos do projeto, não por preferências arbitrárias.